

Biologically-inspired algorithms and models: Report on Local Search

Bartłomiej Woroch 148277 and Maciej Woś 148276

September 13, 2024

1 Introduction

The goal of this report was to compare different approaches to solving the famous Traveling Salesman Problem (TSP). In general, this problem focuses on finding the shortest cycle, connecting all nodes from the initial set, given all the edges between them. Every node can be visited only once. Plenty of real-world applications are related to TSP, such as optimal route planning, logistics, and many more. It belongs to the NP-complete class of problems. The complexity of the algorithm depends on the number of cities and in the worst case it cannot be bounded above by any polynomial. There are many possible ways to solve or approximate the optimal route. This report focuses on exploring how well the local search algorithm (greedy and steepest versions) compares to more crude versions (random and random walk), given similar time interval to explore.

Algorithms are tested over 8 instances of TSP problem found in TSPLIB <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>. Chosen instances: att48, lin105, pr76, berlin52, st70, rd100, kroD100, eil51. They span a range of different numbers of nodes, allowing to test different algorithms in different situations, that are small enough to test them without using more powerful machines than we had access to.

The code is implemented in C++, in such a manner that reuses already existing parts of code. Regarding the most important implementation details:

- every algorithm repeats 50 calculations for every instance,
- to calculate time constraint for `random/random_walk` average of 20 runs of `steepest` algorithm is calculated
- `greedy` and `steepest` runs until better solutions are found
- heuristics implementation:
 - starting from random node find closest to it
 - until the solution is not completed, add another node that is not yet in the solution and is the closest to the beginning or end of current path.

2 Neighborhood

For the experiments 2-OPT operator was used, which exchanges two randomly chosen edges with each other to create the neighboring solution. In a symmetric TSP problem, for such an operator, the neighborhood is of size $n * (n - 1)$, because the full cycle is going to have exactly n edges to choose from and for the next choice only $n - 1$ edges are left.

3 Comparison of the performance

The main analysis of the performance focuses on the running time, number of evaluations (full or partial), and three custom metrics:

3.1 Quality

This metric captures how many times the `obtained_cost` is bigger than `optimal_cost`. To be precise it is of such a form: $(\text{obtained_cost}/\text{optimal_cost}) - 1$. When `obtained_cost` is equal to `optimal_cost` it achieves a value of 0.0. Every bigger `obtained_cost` is going to increase this measure.

3.2 Efficiency

It captures how the algorithm's quality changes over time. Every time an algorithm achieves a final result (e.g. after all its steps for G and S) its quality is calculated. The mean of this list is then divided by the whole time of calculations. It is not the perfect measure and more experiments should be done to investigate its possible improvements, like e.g. some kind of weights.

3.3 Similarity

This report consists of two variants of this measure, `sim_basic` is the ratio between shared edges, between solutions, and the number of edges in the solution. Second, a more sophisticated `sim` measure, also counts the number of shared edges but also tries to favor, those solutions that have shared paths, over a shared number of edges e.g. assuming that we compare two solutions A and B to solution OPT, if solution A and solution B both have 3 common edges to solution OPT, but A have 3 individual edges and B creates a path of the length of 3, solution B is going to be more similar to OPT, than solution A. It is expressed as: $\frac{\#common_edges}{\#common_sub_paths} + \frac{\#common_edges}{\#common_sub_paths}$ (it is important to note, that in `#common_sub_paths`, sub_paths are disjoint to each other)

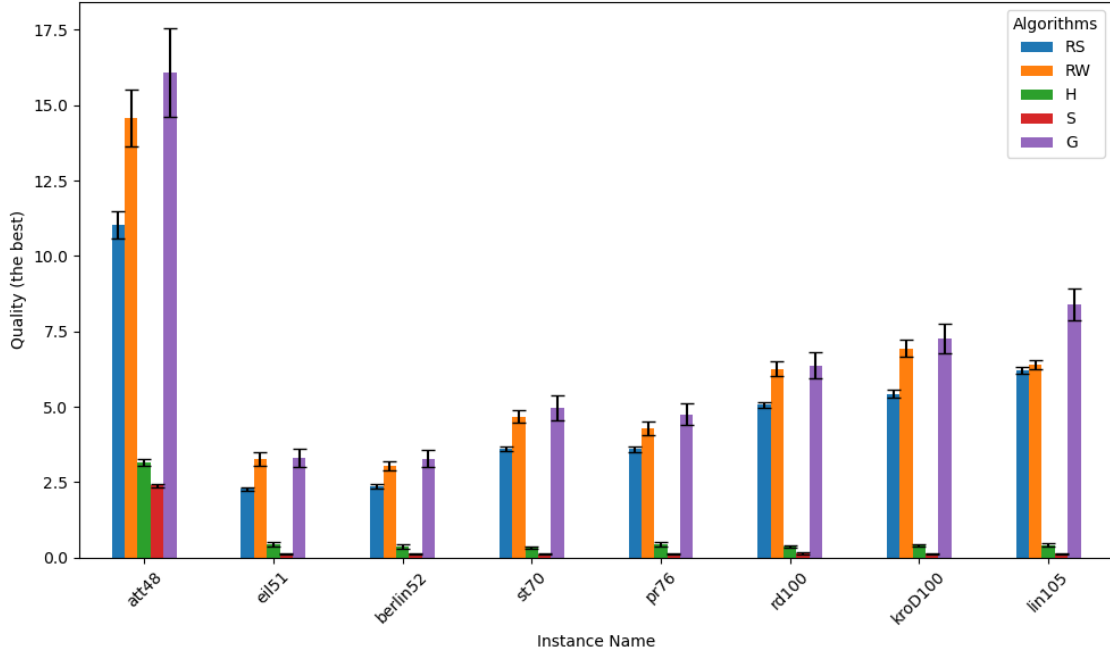


Figure 1: Comparison of the best quality, between algorithms on 8 chosen instances (lower is better). As expected, the steepest achieves the best results on all problems and heuristics closely follow. In most instances, those 2 algorithms achieved better results than $2 * \text{optimal_cost}$. Regarding other observations, greedy did not perform that well (regarding the optimality) Number of repetitions was probably too small (50) and it did not explore the landscape enough. More iterations improve the result, but by a low margin (9, 10).

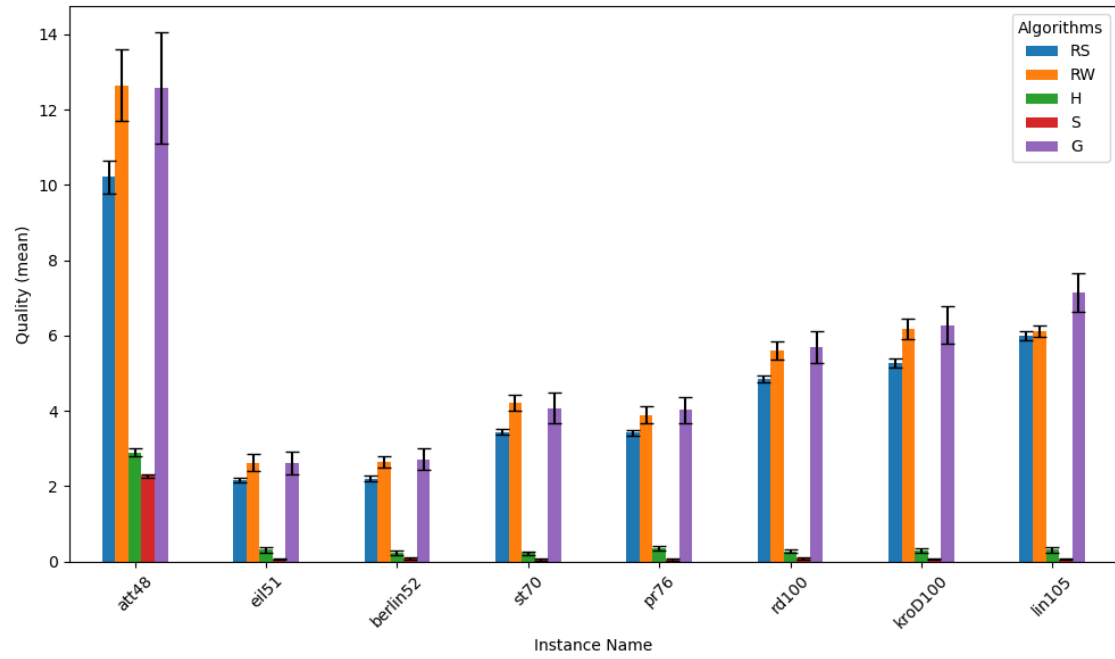


Figure 2: Similar conclusions to the chart 1 can be drawn.

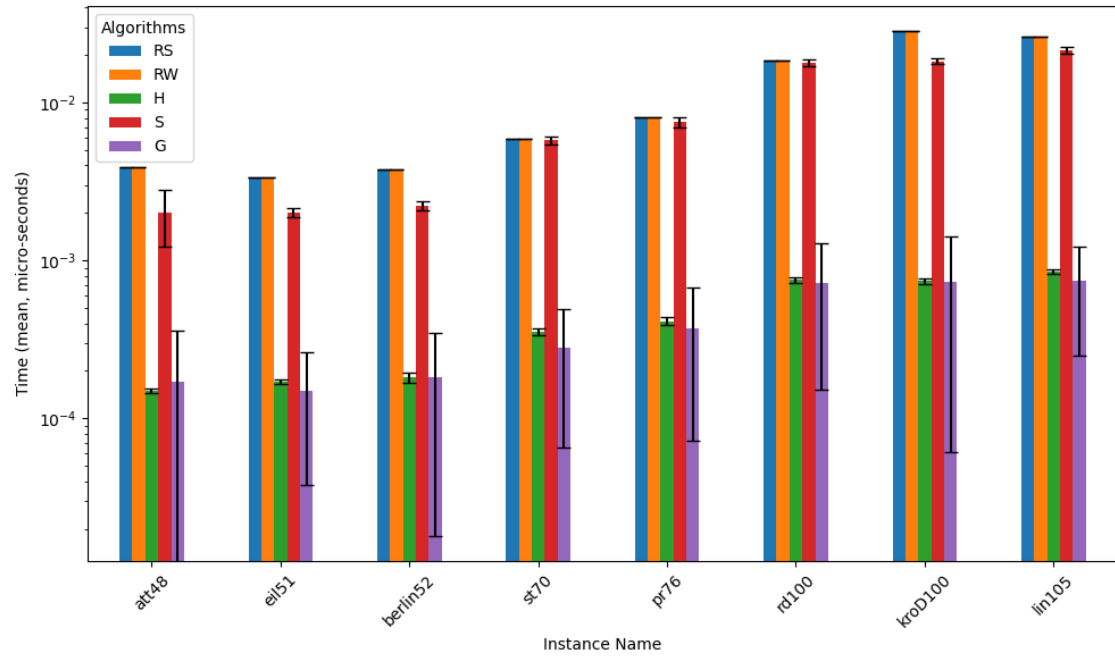


Figure 3: Mean time of all the algorithms over all instances, showed on a logarithmic scale. As expected, the steepest took much longer to compute, compared to the heuristics and greedy approach, which both had similar mean time. Steepest also set the time constraint to random search and random walk approaches.

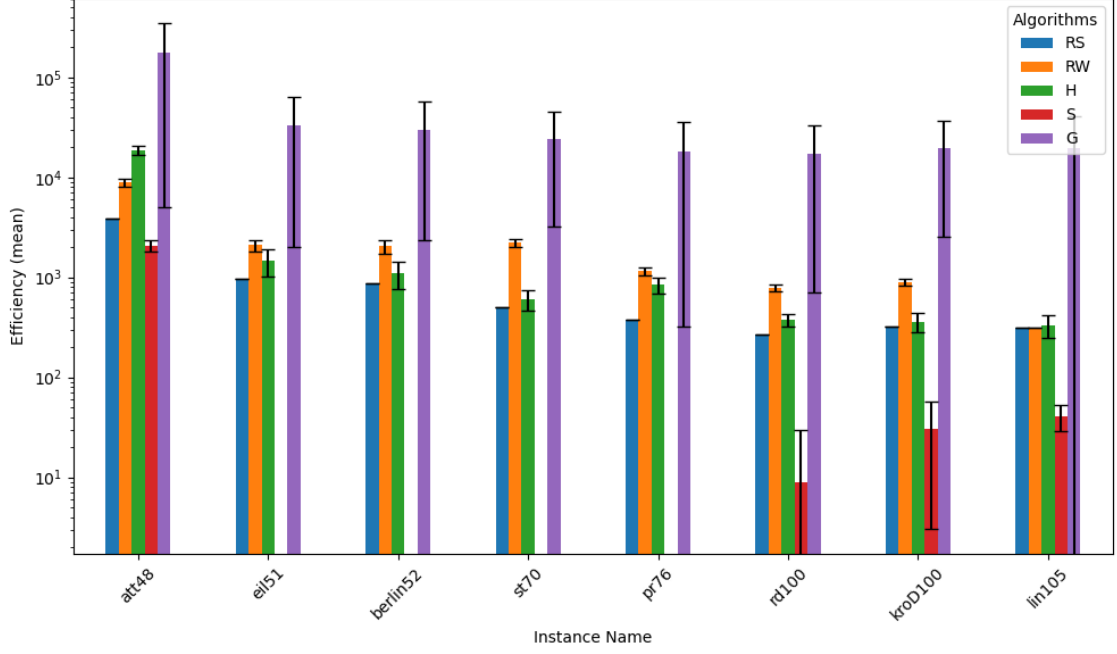


Figure 4: Mean efficiency of the algorithms over all problems, on a logarithmic scale (lower is better). As expected steepest achieves the best efficiency, for smaller problems, becoming smaller than the accuracy of ‘double’ in CPP (indicating that this measure should have some improvements, we were considering it could be a bug in our code, but for those instances, steepest achieves relatively close to the optimal solution) Greedy algorithm performs terrible, not being able to improve quality over time (its mean to be precise) What is more it is also unstable in that manner.

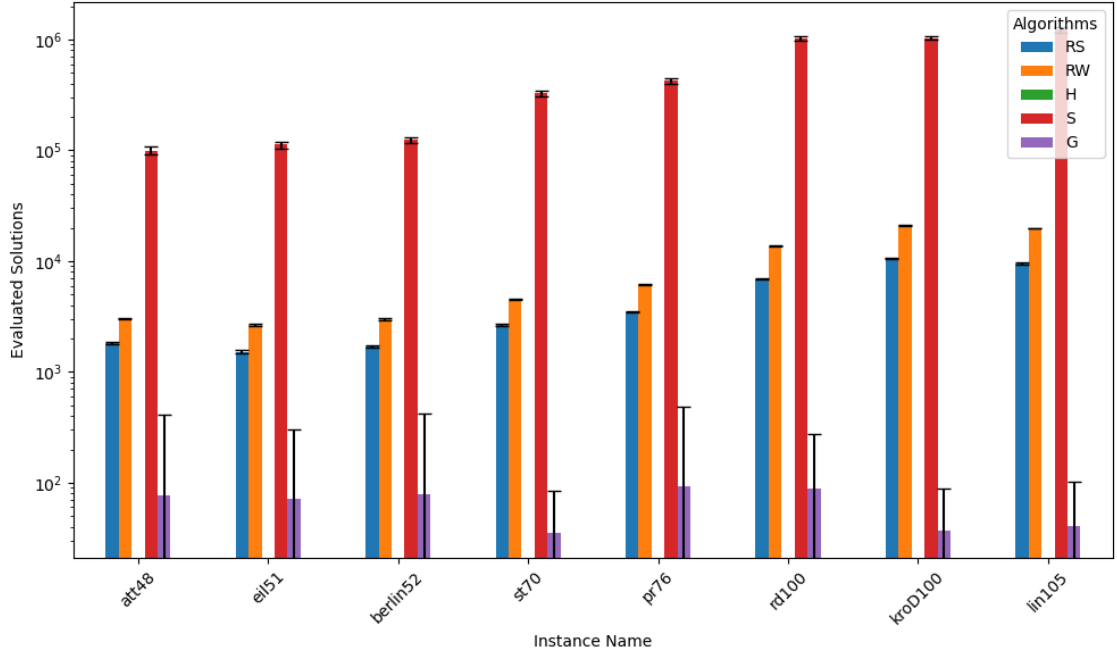


Figure 5: Evaluated solutions of the algorithms over all problems, on a logarithmic scale. Because of its design steepest algorithm always performs the most evaluations. Greedy on the other hand, has a relatively small number of repetitions, but it’s much less stable in that matter. Random search and random walk have a similar number of evaluations, in between results of greedy and steepest algorithm.

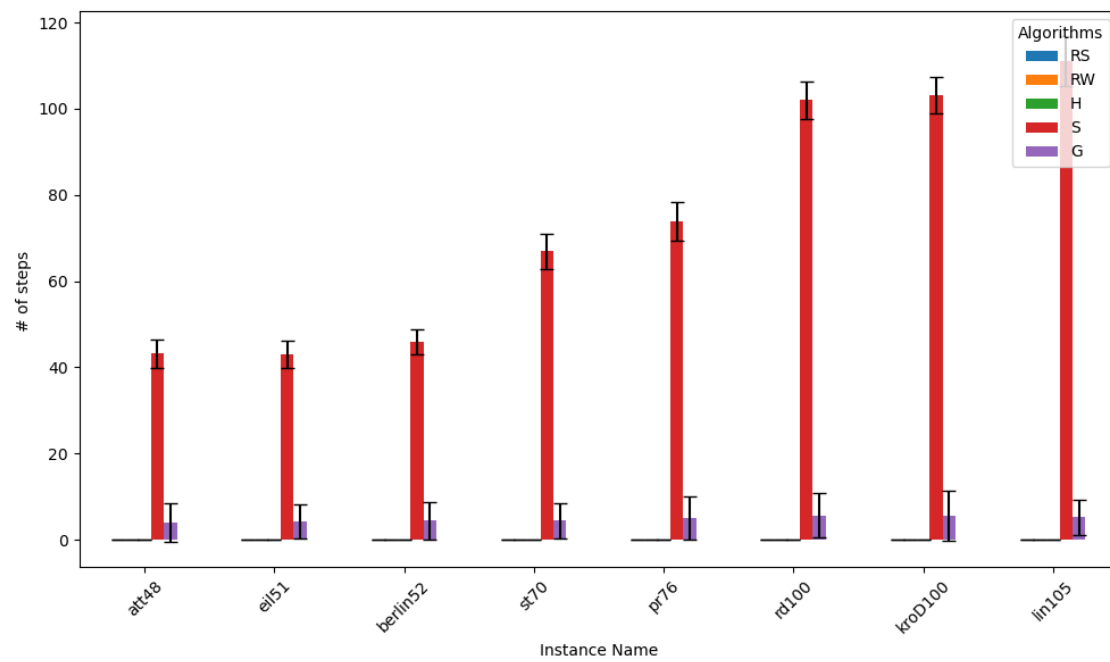


Figure 6: It can be seen on this chart, that the greedy algorithm very quickly (less than 10 steps) finds a local minimum, where it cannot improve anymore, whereas the steepest algorithm needs much more of them, reaching sometimes more than 100 steps.

4 Quality of initial versus final solution

7 and 8 charts show relations between the initial and final solution's costs, with respect to the steepest and greedy algorithm. For the steepest algorithm, the final cost does not depend on the initial solution and reliably converges to a decent answer. On the other hand, the results of greedy depend on the initial solution (Pearson coefficient 0.45). It is worth noting, that some points belonging to the greedy algorithm create a line, bounding cloud of points from above. It is a simple effect of the greedy algorithm, that its final cost cannot be smaller it's than the initial cost. All problem instances had similar results to the chosen ones.

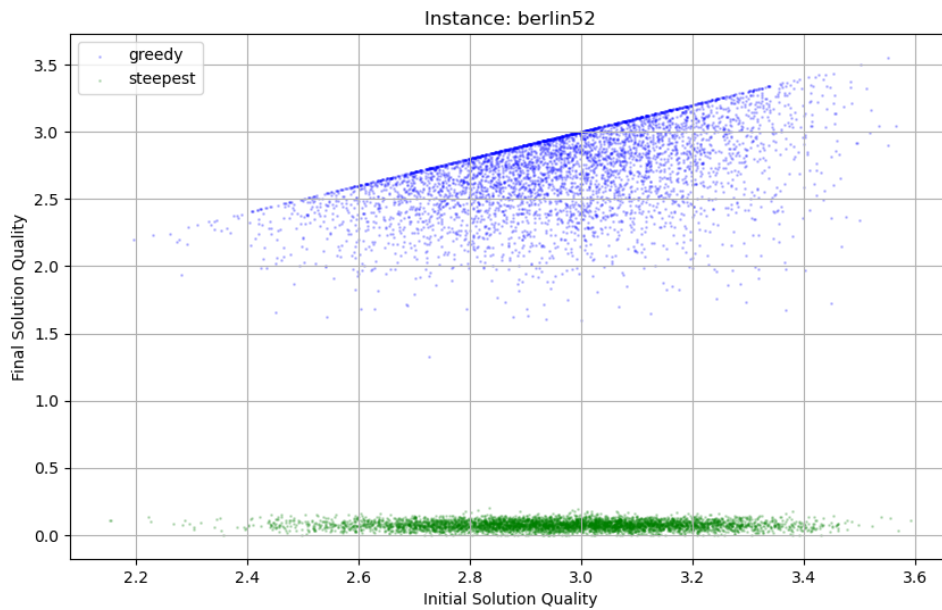


Figure 7: Comparison between initial and final solutions' costs for the steepest and greedy algorithm, on instance berlin52.tsp

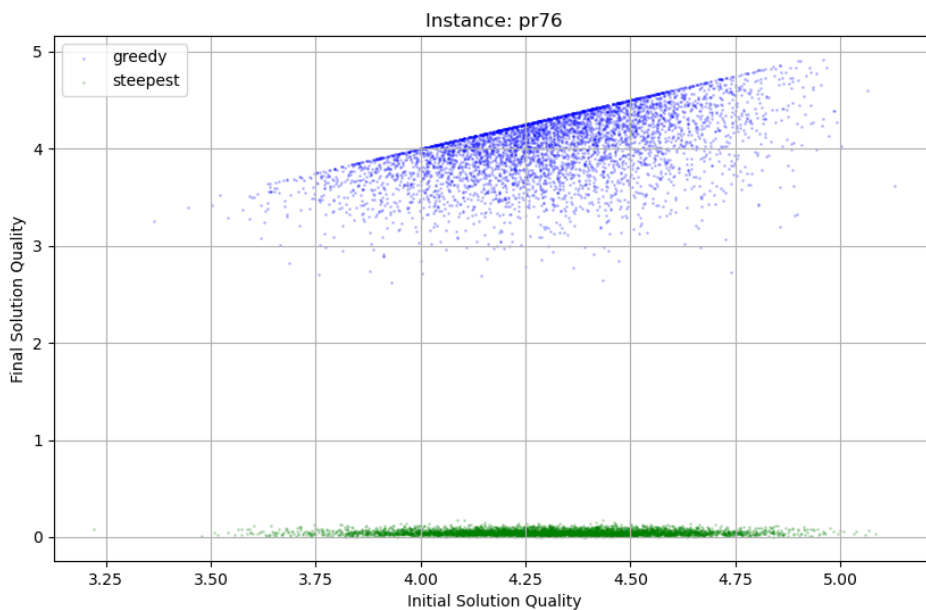


Figure 8: Comparison between initial and final solutions' costs for the steepest and greedy algorithm, on instance pr76.tsp

5 Quality of solutions concerning the number of restarts

Regarding the need for repetitions in calculations of the steepest and greedy algorithm, steepest reliably achieves very good results in the first few iterations, without the need to repeat it more than 10-20 times. Regarding the greedy algorithm, its performance is much worse than the steepest, repeating it more than a few hundred times rarely brings improvements, even if they are not substantial. Its average quickly stabilizes. All problem instances had similar results to the chosen ones.

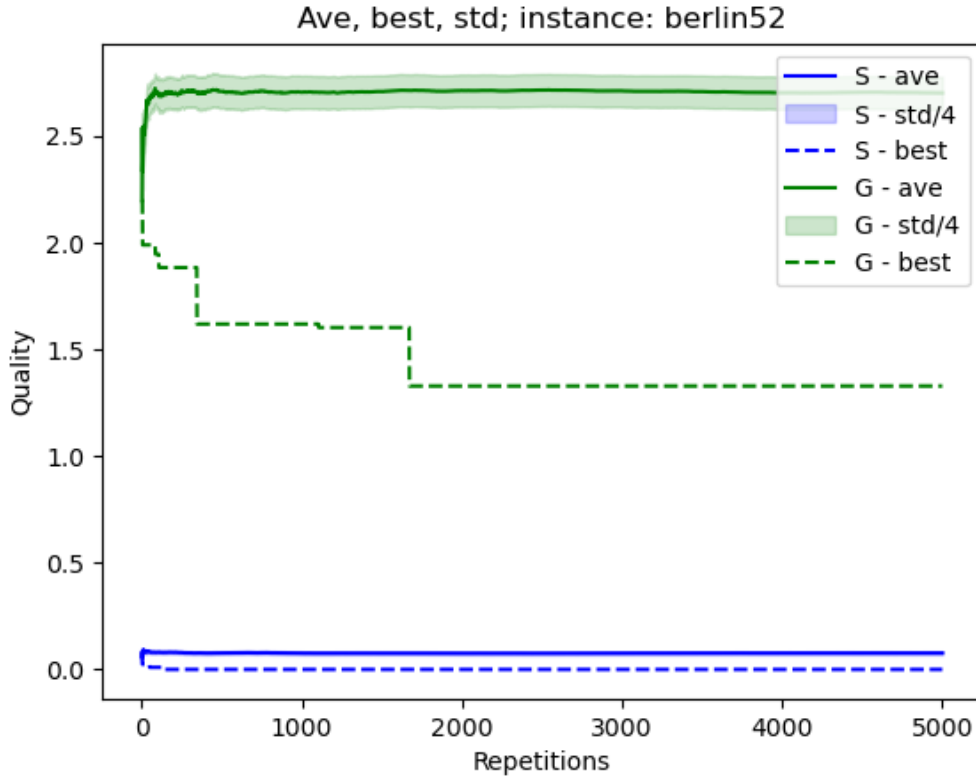


Figure 9: Analysis of quality improvement (smaller is better) with respect to the number of repetitions for greedy and steepest algorithms. Instance: berlin52

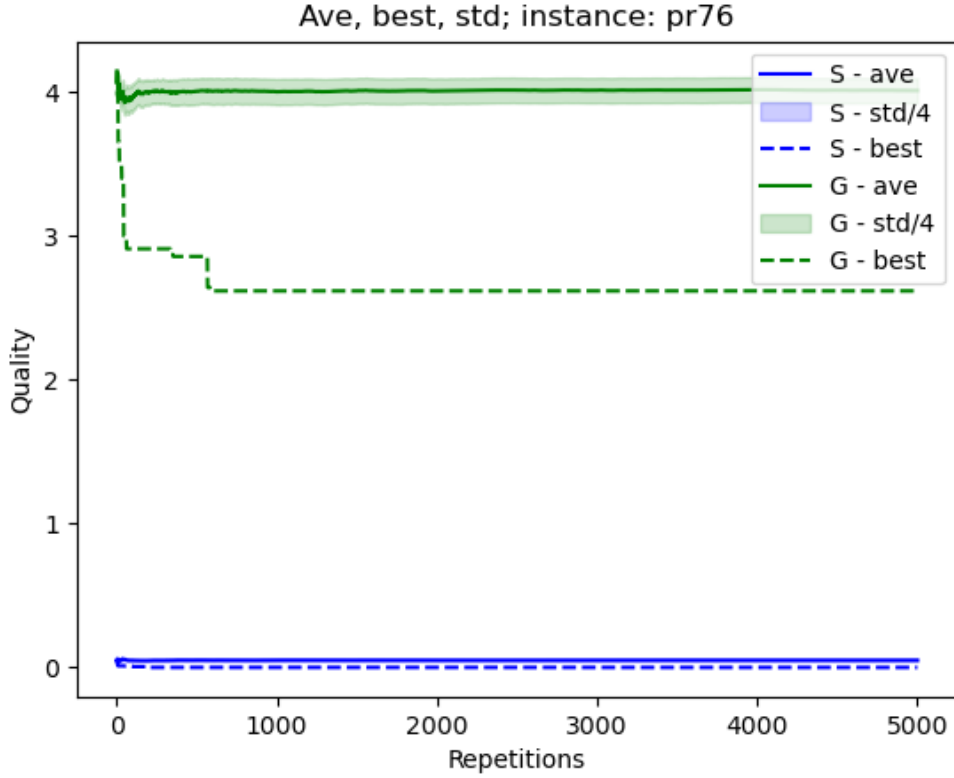


Figure 10: Analysis of quality improvement (smaller is better) with respect to the number of repetitions for greedy and steepest algorithms. Instance: pr76

6 Quality of solutions versus similarity to the best-known optimum

This analysis uses two previously discussed similarity metrics to explore

6.1 Comparison of metrics

Both metrics turned out to provide almost identical results, regarding the clusters of points. Rewarding similar paths, provided marginal differences

6.2 Observations

All algorithms differ mostly in quality, rather than similarity. There are some cases like 13, where its similarity gives us more information. In this example, the steepest algorithm has a similar quality to heuristics but achieves solutions much closer to the known optimum. Random algorithms and greedy approaches have very overlapping clusters, mainly because of their small differences in the obtained quality.

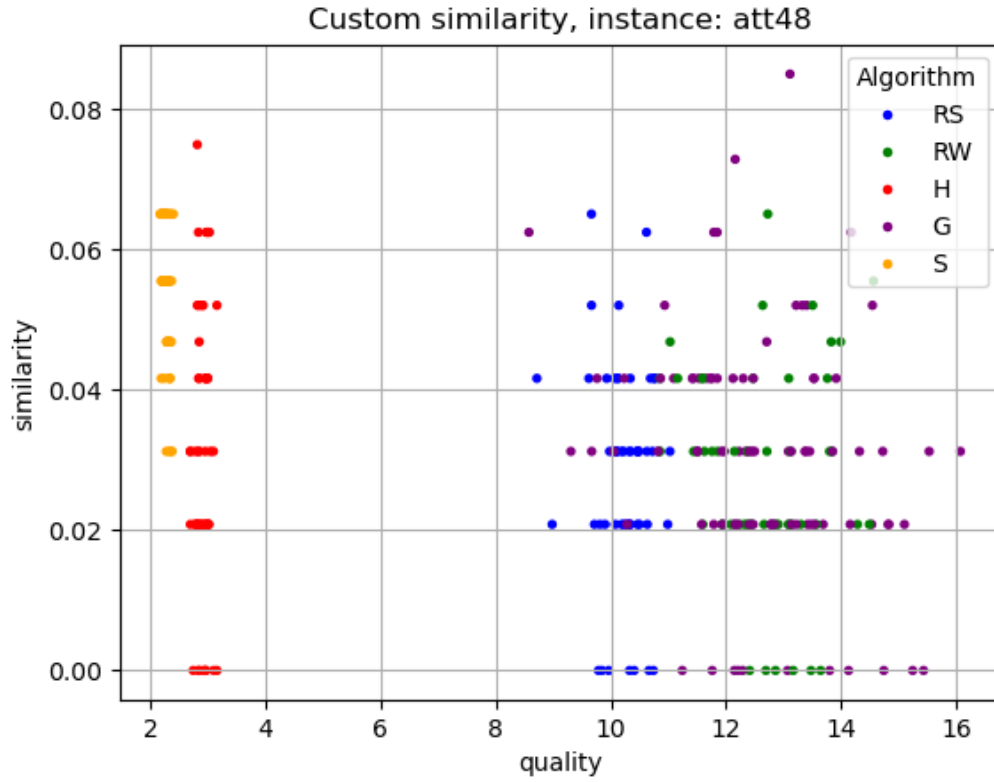


Figure 11: Quality vs. similarity to optimum, using sim metric, over all algorithms on problem att48

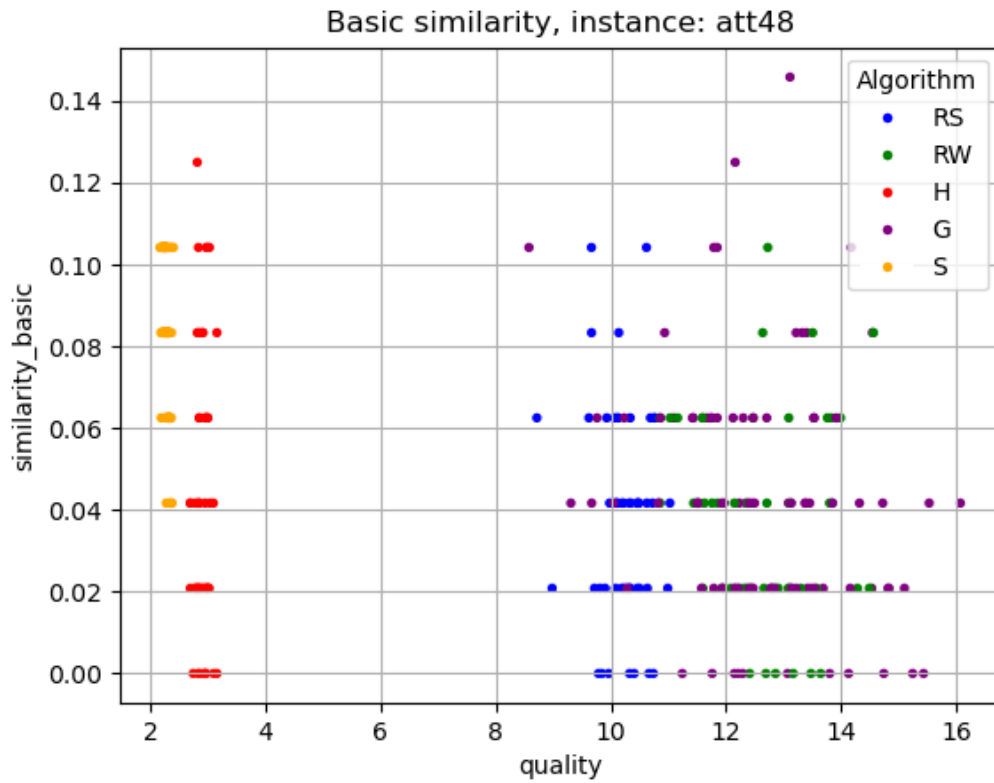


Figure 12: Quality vs. similarity to optimum , using sim_basic metric, over all algorithms on problem att48

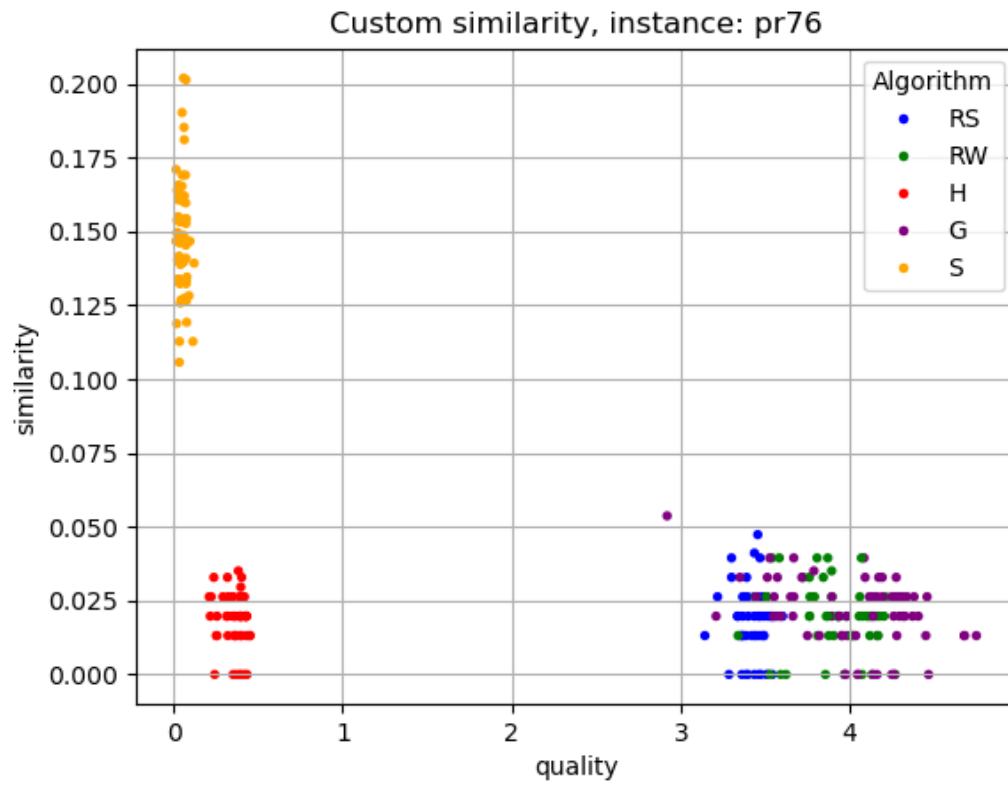


Figure 13: Quality vs. similarity to optimum, using sim metric, over all algorithms on problem pr76

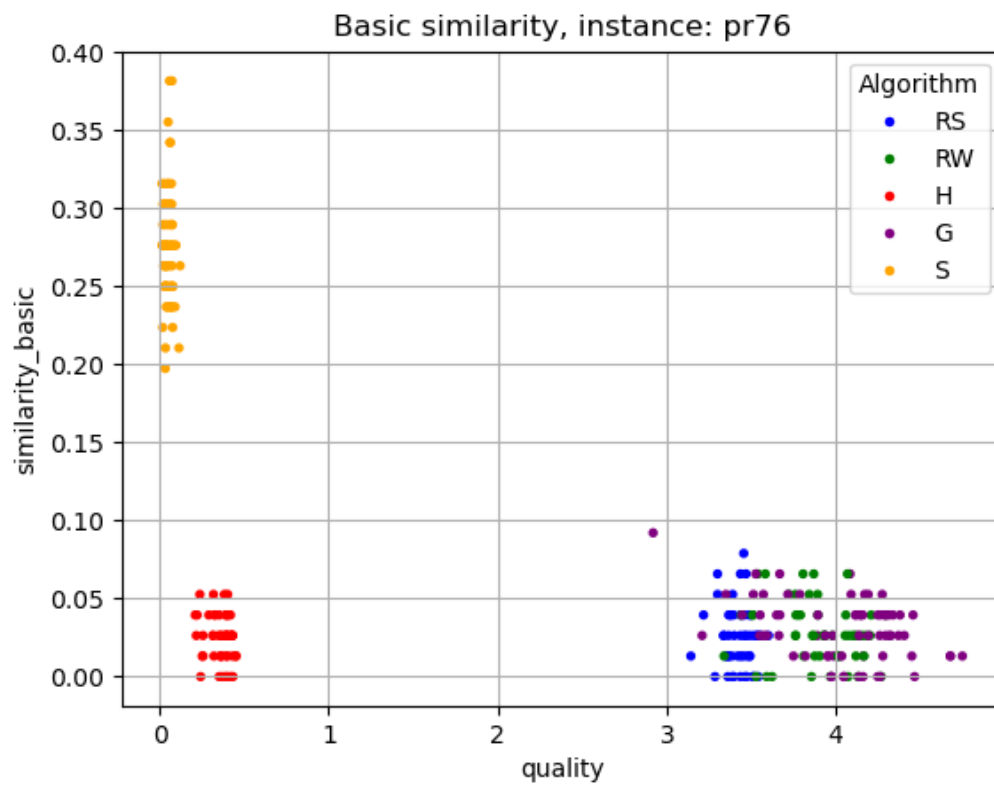


Figure 14: Quality vs. similarity to optimum , using sim_basic metric, over all algorithms on problem pr76

7 Conclusions from the experiments

Most of the algorithms performed to their expectations. Steepest achieved the best quality, at the same time being the slowest of them all. Heuristic, also created decent solutions, closely following the results of the steepest. At the same time, it created completely different solutions, compared to the steepest solutions 14. Random search was a little bit worse than Random walk, but both approaches could not find decent solutions even though they were given similar execution time to the steepest algorithm. Last but not least, there is a case of a greedy algorithm. Unfortunately, it could not find decent solutions, having similar quality to the random solutions, but with even higher standard deviations. 9 and 10 show that more repetitions achieve better costs, but they are far from the costs of the steepest algorithm. It is probably because the fitness landscapes of considered problems, requiring more exploration to achieve better results, that random and greedy algorithms could not deal with.

8 Difficulties encountered

The main issue we have been struggling with is the time management and mental barrier preventing us from working on this project. The workload of other exercises, projects, and other master's degree curricula activities had us completely overwhelmed. It was necessary to abandon, for the time being, part of our duties to fulfill them, at least to some degree. To be honest, only after a 2-month summer break, we have gathered enough distance to tackle this project without fear or anxiety.

Another issue we have encountered is working with a new for us programming language. Even though we understand the basic principles of it and have a little bit of experience in CPP, it startled our progress quite a bit. Also, plenty of habits and good practices that we were not aware of turned out to be a true blessing (e.g. running the compiler with `-O3` flag, forces it to optimize, with respect to time, which improved the computation 4x times)

9 Justification of the introduced improvements, suggestions for improvements, and their expected effects

9.1 Introduced improvements

- greedy and steepest, while evaluating neighbors, calculates only the change in the solution, preventing time loss in calculating the whole cost
- managing TSP instances via custom `Problem` class, in which computations of neighborhood and distance array, needed to be done only once
- passing reference to the problem object was one of the biggest time-saving improvements we have made. It was oblivious to us that it is possible (lack of intermediate CPP programming skill) After the change some instances with around 100 nodes, speed up approximately 1000 times.

9.2 Suggested improvements

- **Limiting the neighborhood size:** by pre-calculating for each node its own sorted list of neighbors, we would save time on exploring the neighborhood in the steepest and most greedy algorithm. At the same time, it would be wise to introduce into that list some kind of randomness (e.g. randomly swap 10% of nodes with each other or any other not deterministic ordering, for which distance is only a suggestion to follow) to avoid getting stuck in the local optima and allow for better exploration.
- **Better custom metrics:** even though a considered time has been spent on crafting them, we believe that better and more informative metrics can be crafted. One of the possible similarity measures that we did not explore (but we considered using it) is to, for each node, calculate the difference between its place in solution A and solution B, sum it up, and scale it over the size of the problem. (Of course, because of the nature of the cycle, it would be necessary to repeat those calculations for each possible rotation and take a minimum of it)

10 Stimulated annealing and Tabu search implementations

10.1 SA implementation

- Initial temperature is set to be $-ave_diff/\ln(0.95)$, where `ave_diff` is the average change in cost, over all possible edge swaps of the initial solution. Parameter `K` is equal to 5, `L` is $\sqrt{no_nodes}$, and `alpha` was experimentally chosen to be 0.96.
- run `while` loop, as long as the temperature is higher than 0.01 or the number of temperature changes is smaller than $L * K$
 - set `i` to 0 and begin for loop, until $i < L$
 - `SA_step`
 - * find the best possible edge swap from the considered solution
 - * if it achieves a better score make a swap
 - * do it otherwise, with probability proportionally to the temperature
 - if the better solution was found, overwrite old solution, update new cost, and change `i` to 0
- update the temperature and increase the `SA_steps` counter by one.

As required from the exercise, `initial_temperature` is created in such a way, that it accepts 95% of the neighborhood. We have tried to experimentally find the best `alpha`, but it is impossible to find one value that fits all of the problems. For some problems, a particular value can yield very good quality, whereas the same value for other problems can drastically decrease its quality. We have chosen a value of 0.96, which is moderately positive for all cases.

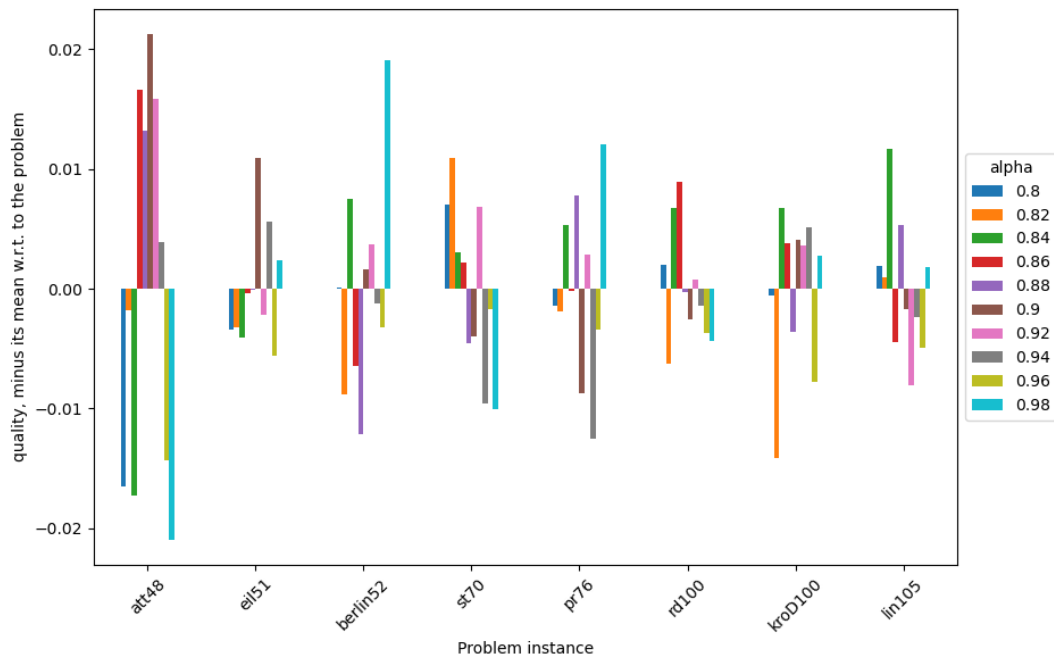


Figure 15: Quality of different values of `alpha` for all instances, averaged over 20 runs. Results are centered around 0.0 ($result - mean_quality_of_the_problem$). It is impossible to find one `alpha` that would fit all the problem instances.

10.2 TS implementation

- a list of banned edge swaps of length $0.3 * no_nodes$ is kept in memory and updated every time a new swap is updated
- `TS_step`
 - search the neighborhood, looking for the best swap

- if swap creates cost, that is better than currently the best cost (aspiration) accept the swap
- or if the swap is better than currently the best swap and it is not in the banned edges list, accept the swap
- apply the swap
- repeat **TS_step** until there is no improvement for $3 * \text{no_nodes}$ iterations

We have been experimenting with different lengths of tenure 16 sizes and the maximal number of iterations 17, but in both cases it is impossible to find one value that fits all solutions. Finally, the length of tenure is 30% of the number of nodes, and the maximal number of repetitions is $3 * \text{no_nodes}$.

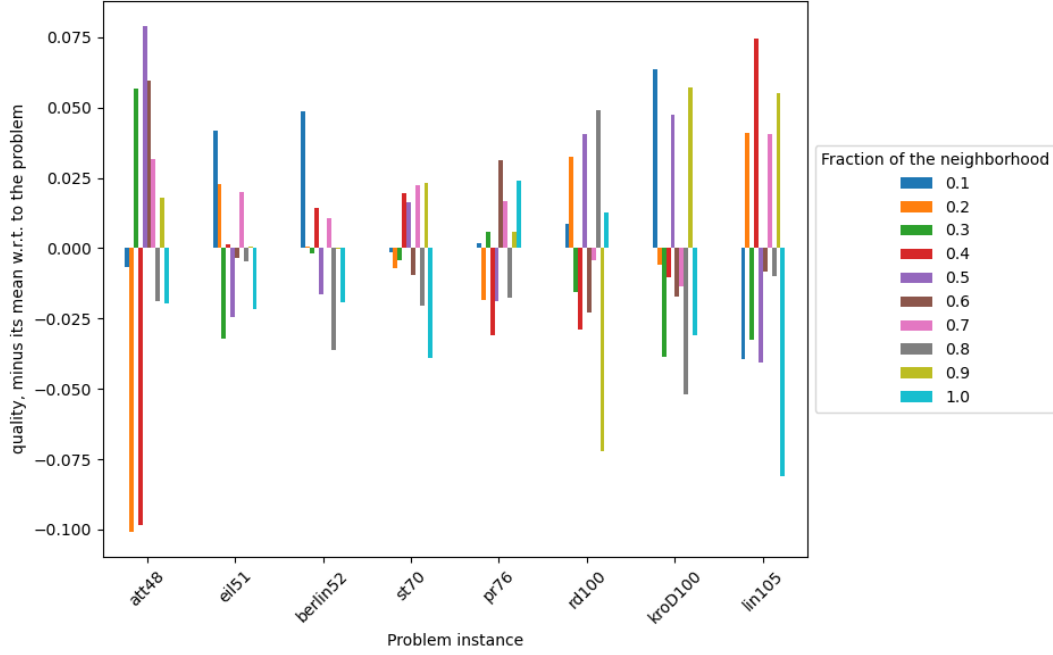


Figure 16: Quality of different lengths of tenure table, for all instances, averaged over 50 runs. Results are centered around 0.0 (*result - mean_quality_of_the_problem*) It is impossible to find one tenure length that would fit all the problem instances.

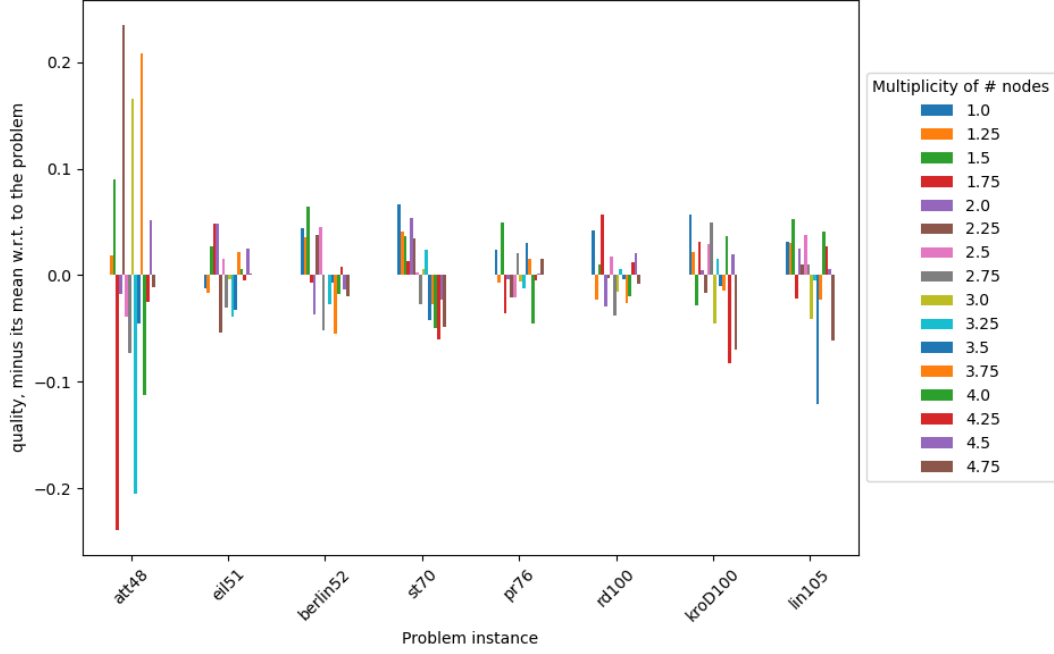


Figure 17: Quality of different maximal number of repetitions, expressed as multiplicities of `no_nodes`, for all instances, averaged over 50 runs. Results are centered around 0.0 (*result – mean_quality_of_the_problem*) It is impossible to find one value that would fit all the problem instances.

11 Performance of all algorithms

Stimulated annealing (SA) performed similarly to the steepest algorithm, regarding the quality. As expected it needed more time and solution evaluations. It outperformed the heavily steepest algorithm in efficiency metric. It is a rather surprising observation. Frankly, We cannot be sure why this is the case. SA is the steepest approach, which spends more time exploring the landscape of the problem, so it should spend more time (which it does 21) finding better solutions (the difference to the steepest is marginal) So in theory SA should not have better efficiency than steepest. The most reasonable solution why, we have come up with is that our **efficiency** metric is not well refined, and for future experiments, we should redesign it.

Tabu search (TS) was just a bit worse in quality than SA, but it got much better results regarding mean time, and (lower) number of evaluated solutions. Regarding efficiency, it is better than the steepest, but could not beat the SA algorithm. Unfortunately similar argument used in SA can be used, that the efficiency metric should be redesigned for better analysis.

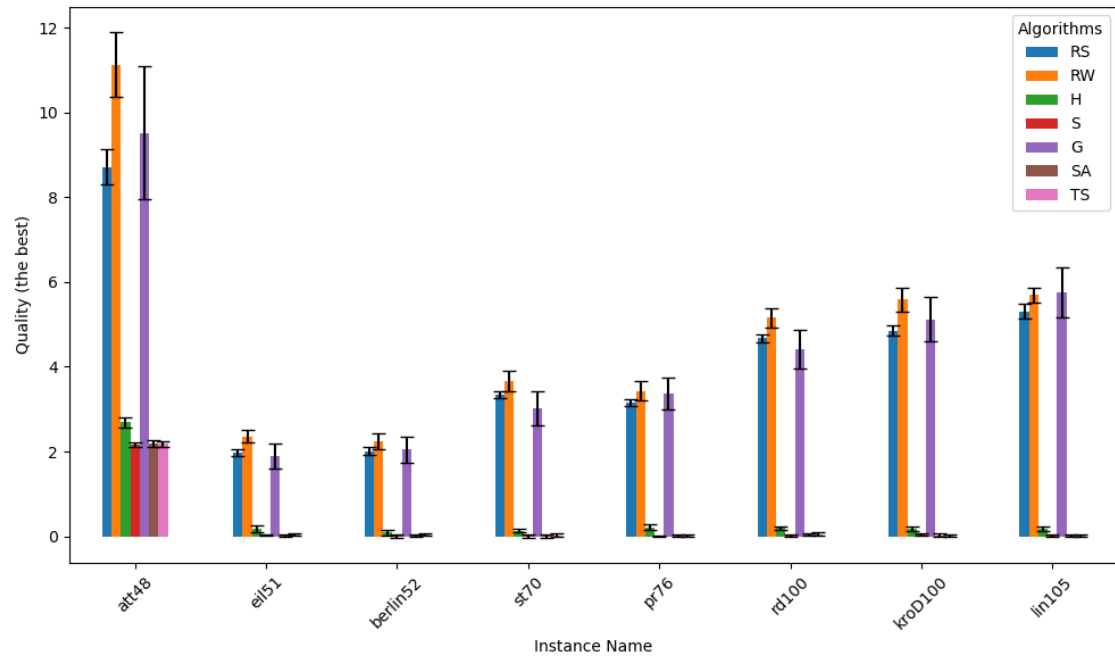


Figure 18: The best quality of all algorithms over all problem instances (lower is better)

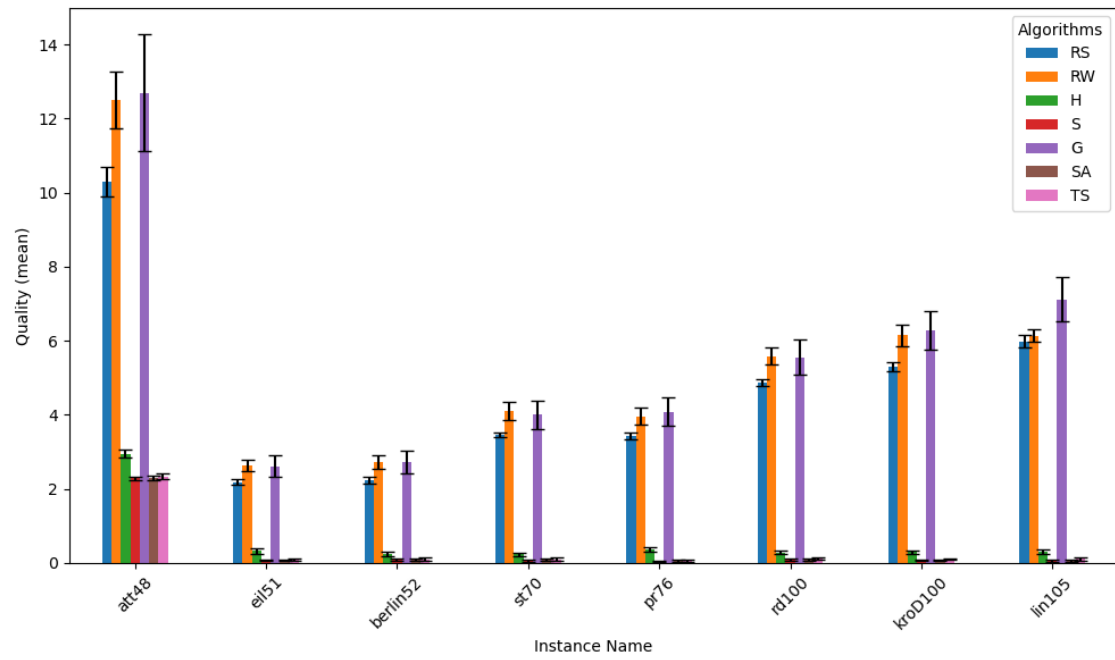


Figure 19: The mean quality of all algorithms over all problem instances (lower is better)

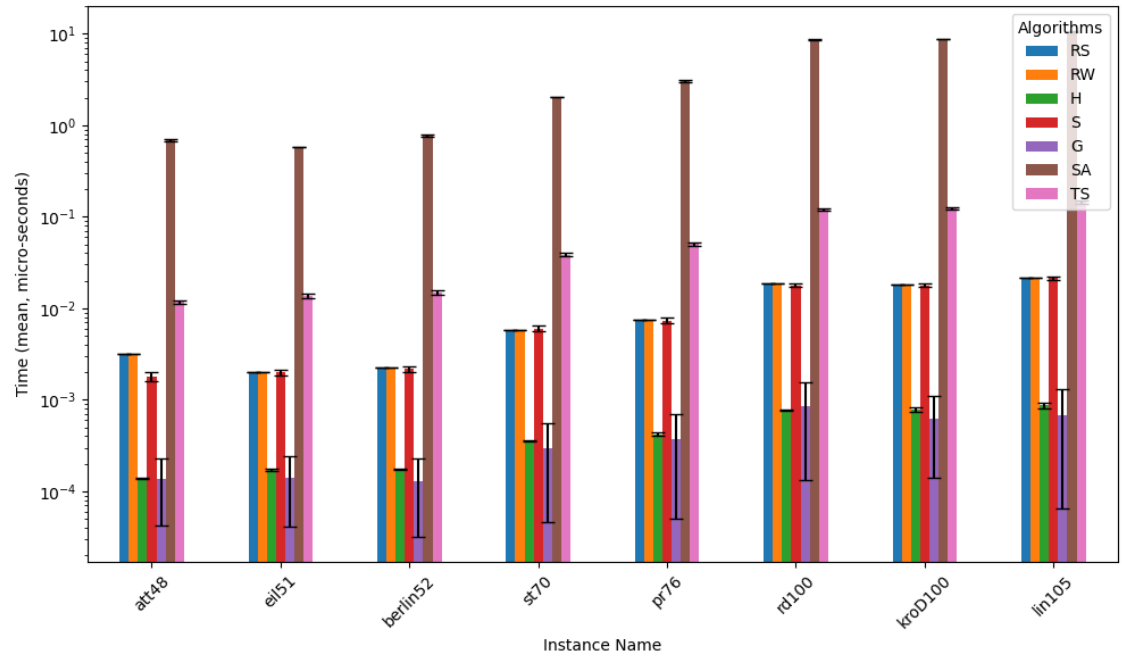


Figure 20: Mean time of all algorithms, in microseconds, over all problem instances, on logarithmic scale

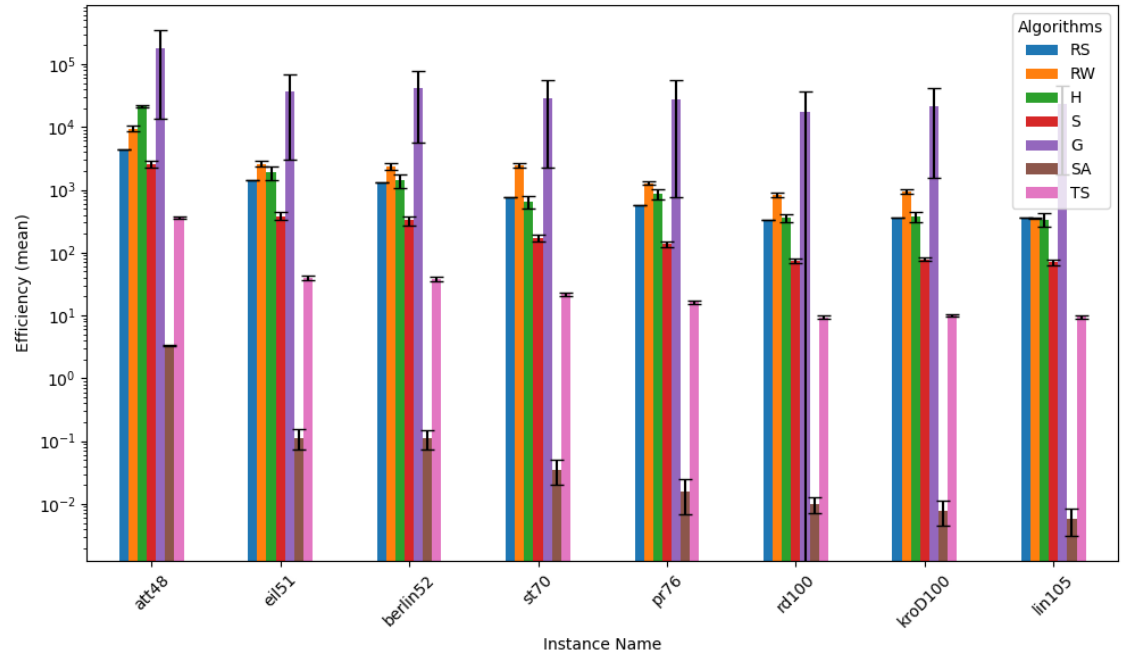


Figure 21: Mean efficiency of all algorithms over all problem instances, on a logarithmic scale (lower is better)

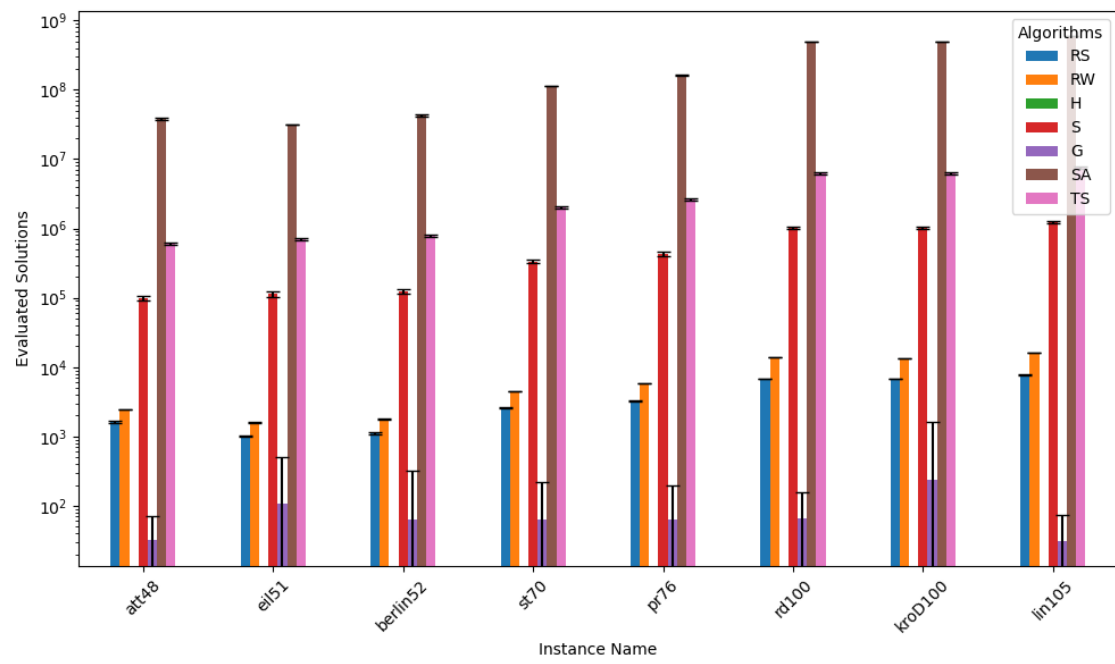


Figure 22: Number of evaluated solutions for all algorithms, overall problem instances, on a logarithmic scale (lower is better)

12 Final conclusions

The performed experiments showed the main differences between different approaches to fitness landscape exploration and evaluation. Even though many patterns can be observed e.g. trade-offs between execution time and achieved solution, it always boils down to analysis of particular problems. Even though they can have a similar number of nodes, their distribution in their space, can drastically change the fitness landscape, making it impossible to find one algorithm (or their parameters) that fits all scenarios. Even though metrics such as efficiency should allow for better judgment, in the case of this report, they are not perfectly designed and should be redesigned. Nevertheless, the steepest, simulated annealing and tabu search, achieved the best results regarding quality, with a compromise of being slower than other approaches.